Porto. TechHub

P.PORTO

AI-POWERED SOFTWARE ENGINEERING

| ADRs |

VITOR MANUEL SANTOS

PROGRAMAÇÃO

- Architecture Decision Record
- Estrutura e Boas Práticas na criação e manutenção de ADRs
- Utilização do GitHub & GitHub Pages como ferramenta de organização de ADRs

VITOR MANUEL SANTOS

ADRs

VITOR MANUEL SANTOS

ADR

- **ADR** significa **Architecture Decision Record**.
- É um registo curto de uma decisão arquitetural e da respetiva justificação.
- O objetivo é deixar explícito o que foi decidido, porquê, que alternativas existiam e quais as consequências. A coleção de ADRs de um projeto forma o seu decision log.

Um ADR não documenta tudo sobre a arquitetura, documenta uma decisão importante e o racional por trás dela.

VITOR MANUEL SANTOS

ADR

- Sem ADRs, é comum a equipa lembrar-se da decisão, mas esquecer-se do motivo.
  - Passado algum tempo, alguém pergunta:
  - Porque escolhemos esta arquitetura?
  - Porque não seguimos outra alternativa?
  - Este trade-off foi consciente ou acidental?
- Os ADRs ajudam a:
  - Preservar contexto
  - Justificar escolhas
  - Registar trade-offs
  - Facilitar onboarding
  - Apoiar evolução futura da arquitetura

ADR

- Segundo o portal ADR (https://adr.github.io/), uma Architectural Decision é uma escolha de desenho de software, devidamente justificada, que responde a um requisito funcional ou não funcional com significado arquitetural.
- Ou seja: não é uma decisão trivial, é uma decisão com impacto relevante no sistema.

ADR

- Exemplos que fazem sentido em ADR:
  - monólito modular vs microserviços
  - REST vs messaging
  - PostgreSQL vs MongoDB
  - autenticação com JWT vs sessão
  - adoção de CQRS ou não

ADR VS MADR

- É importante distinguir:
  - **ADR** = o conceito de registar decisões arquiteturais
  - **MADR** = Markdown Architectural Decision Records, um template estruturado para escrever ADRs em Markdown
- O **MADR** foi criado para captar decisões de forma leve, estruturada e prática, mantendo foco nas decisões arquiteturalmente significativas. Pode ser usado apenas com ficheiros Markdown, sem necessidade de software complexo.

NYGARD VS MADR

- O portal ADR destaca várias famílias de templates, incluindo:
  - **Nygard ADR**
  - **MADR**
  - **Y-Statements**

NYGARD VS MADR

- O template Nygard é a estrutura clássica e mais simples, normalmente com:
  - Title
  - Status
  - Context
  - Decision
  - Consequences

NYGARD VS MADR

- O MADR expande esta ideia e costuma incluir uma análise mais explícita de:
  - Contexto e problema
  - Opções consideradas
  - Decisão tomada
  - Consequências
  - Por vezes metadados como decision makers, consulted, informed e status

MADR

- O MADR dá bastante importância às opções consideradas e aos respetivos prós e contras, para que o leitor perceba melhor o racional da decisão. O próprio portal refere que essa análise de trade-offs é crucial para compreender porque foi escolhida uma solução e não outra.
- Além disso, o MADR oferece versões diferentes do template, incluindo:
  - full
  - minimal
  - annotated
  - bare

MADR

- Na prática, os ADRs costumam ser guardados como ficheiros Markdown no repositório, por exemplo numa pasta como docs/decisions. O MADR recomenda precisamente esta abordagem, com ficheiros numerados e nomes com hífen, como:
  - 0001-use-modular-monolith.md
- A vantagem é simples:
  - Ficam versionados com o código
  - Podem ser lidos no GitHub
  - Podem integrar documentação do projeto
  - Tornam-se parte natural do ciclo de desenvolvimento

MADR – TOOLING

- O portal ADR tem uma secção própria de Decision Capturing Tools e refere várias abordagens e ferramentas para criar e gerir ADRs. Também deixa claro que há múltiplos templates e práticas, não apenas uma única forma oficial de o fazer.
- Ainda assim, para começar, muitas equipas nem precisam de tooling pesado:
  - **Markdown + template + Git** costuma ser suficiente.
- O portal MADR refere explicitamente que o template pode ser usado em qualquer editor de texto.

ADR

- Ao usar esta estrutura:



- A pasta docs/decisions está alinhada com a recomendação do MADR, e o nome dos ficheiros no formato NNNN-title-with-dashes.md também é o padrão sugerido.

ADR – CRIAR REPOSITÓRIO

- Entra no GitHub.
- Clica em New repository.
- Escolhe o owner.
- Dá um nome ao repositório, por exemplo:
  - adr-demo
  - software-architecture-notes
  - pulma-porto-architecture
- Ativa o README.
- Clica em Create repository.
- O GitHub documenta este fluxo de criação de repositórios pela interface web:
  - https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository

ADR – CRIAR A ESTRUTURA DOS ADRs

- Depois de criares o repositório, adiciona a pasta:
  - docs/decisions
- Podes fazer isso diretamente no GitHub criando ficheiros com esse caminho, ou localmente no teu computador.
- Localmente:
  - git clone <URL_DO_REPOSITORIO>
  - cd adr-demo
  - mkdir -p docs/decisions

ADR – CRIAR PRIMEIRO ADR

- O primeiro ADR costuma registar a própria decisão de usar ADRs no projeto. Isso é prática comum nas ferramentas e templates ADR, e o adr-tools, por exemplo, cria precisamente esse registo inicial quando fazes adr init.
- Seguidamente cria o ficheiro:
  - docs/decisions/0000-use-architecture-decision-records.md

ADR – CRIAR PRIMEIRO ADR

- Conteúdo sugerido:

```
# 0000 - Use Architecture Decision Records

## Status

Accepted

## Context

O projeto precisa de registar decisões arquiteturais importantes de forma simples, rastreável e versionada.

## Decision

Serão usados Architecture Decision Records (ADRs) em ficheiros Markdown, guardados em `docs/decisions`.

## Consequences

- As decisões passam a ficar documentadas no repositório
- O contexto e os trade-offs ficam preservados
- Novos elementos da equipa conseguem perceber melhor o racional técnico
```

ADR – CRIAR PRIMEIRO ADR

- O portal ADR explica que um ADR captura uma decisão arquitetural e o seu racional, e o conjunto dos ADRs constitui o ***decision log*** do projeto.

ADR – CRIAR MAIS ADRs

- Agora vais criando um ficheiro por decisão relevante.
- Exemplos:
  - docs/decisions/0001-use-modular-monolith.md
  - docs/decisions/0002-use-postgresql.md
  - docs/decisions/0003-use-rest-between-clients-and-api.md

ADR – TEMPLATE

0001 - TÍTULO DA DECISÃO

Status
Proposed

Context
Qual é o problema? Em que situação estamos? Que restrições existem?

Options Considered
- Opção A
- Opção B
- Opção C

Decision
Escolhemos a opção X porque...

Consequences
- Consequência positiva 1
- Consequência positiva 2
- Trade-off / limitação 1

ADR – COMMIT & PUSH

- Se estiveres a trabalhar localmente:
  - git add .
  - git commit -m "Add initial ADR structure“
  - git push origin main
- Depois disso, os ADRs ficam no repositório como ficheiros Markdown normais.

ADR – VISUALIZAR OS ADRs

- Ao abrires um ficheiro .md no GitHub, ele é renderizado automaticamente como Markdown formatado. O GitHub documenta o uso de README e Markdown como forma normal de documentar repositórios.
- Na prática, fazemos:
  - Abrir o repositório
  - Entrar em docs/decisions
  - Clicar num ficheiro, por exemplo 0001-use-modular-monolith.mdE ele aparece já legível, com títulos e listas.
- Limitação: Fica funcional, mas não é uma navegação muito bonita se tiveres muitos ADRs.

ADR – MELHORAR A NAVEÇÃO C/ ÍNDICE

- Para facilitar, cria um ficheiro:
- docs/decisions/README.md
- Com o seguinte conteúdo:

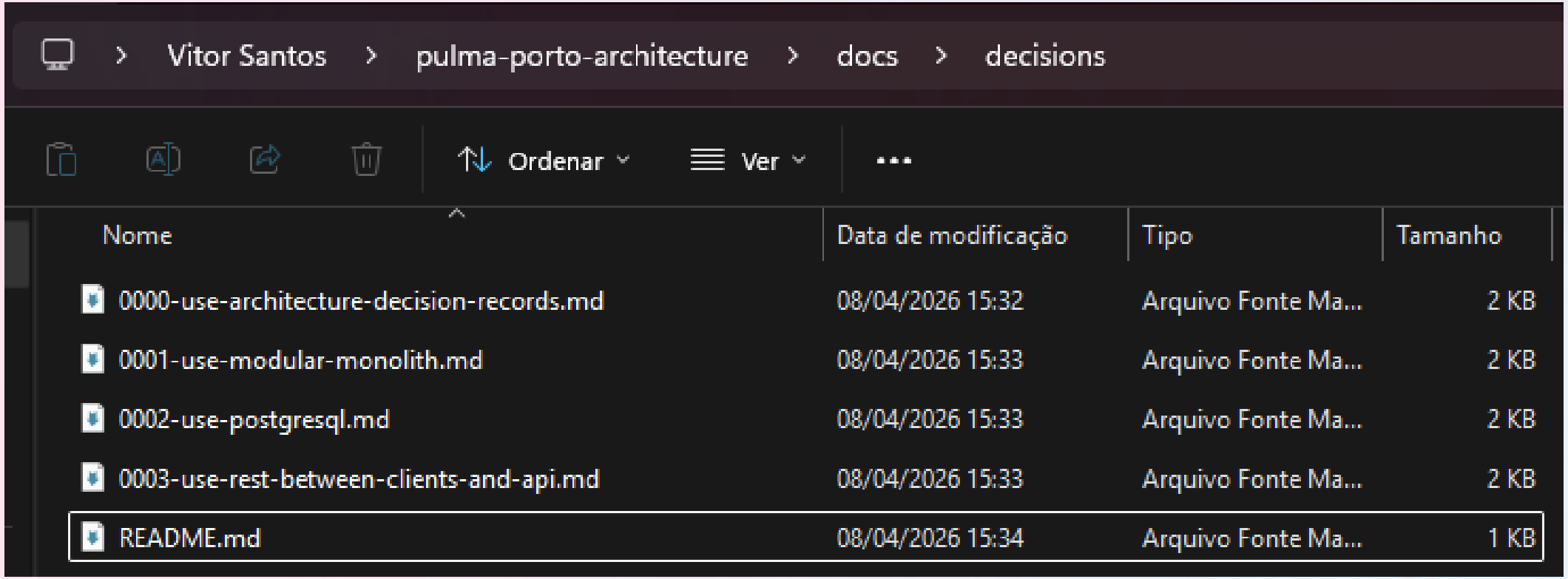
```
# ADRs do Projeto

## Lista de decisões

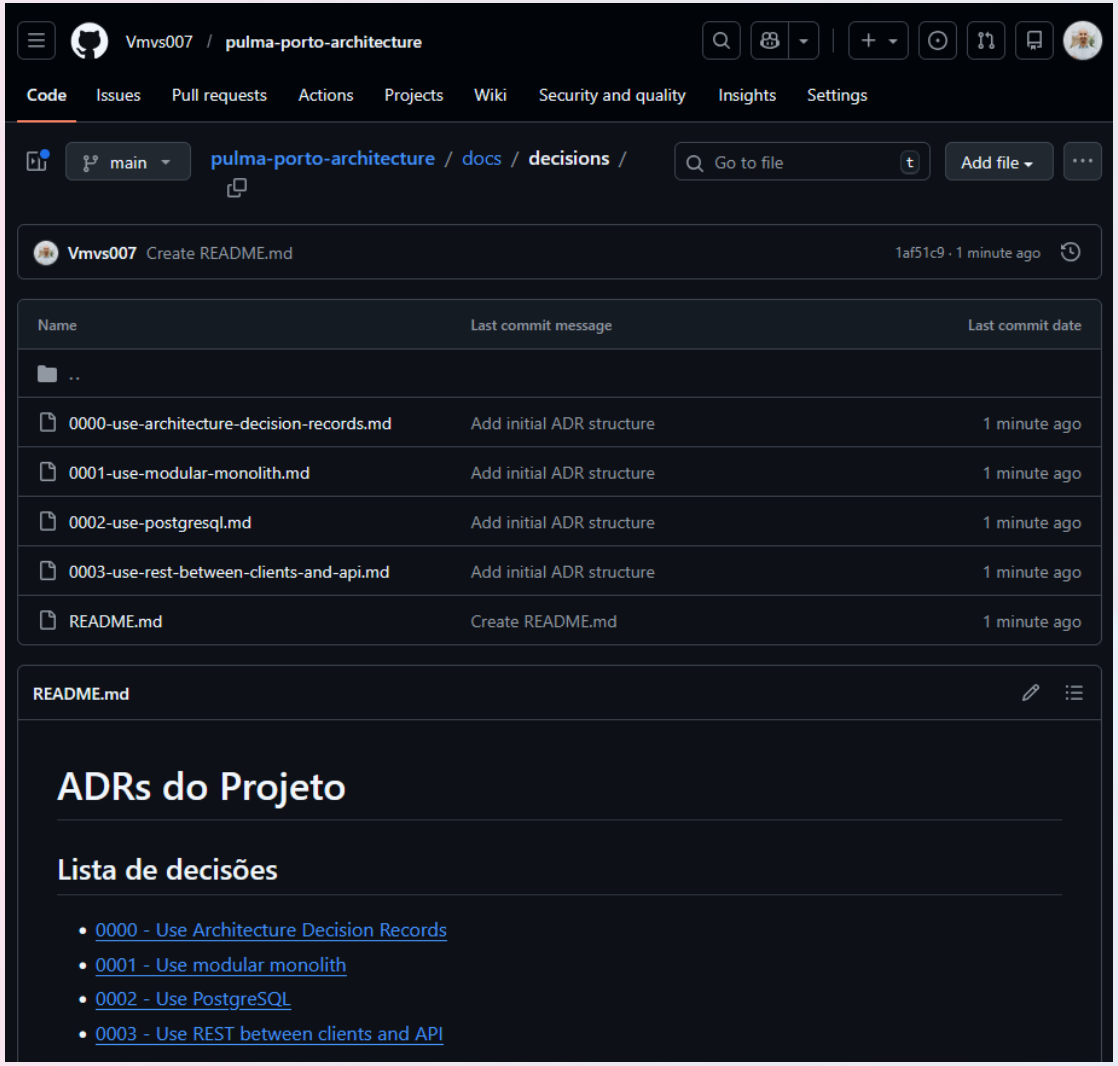
- [0000 - Use Architecture Decision Records](./0000-use-architecture-decision-records.md)
- [0001 - Use modular monolith](./0001-use-modular-monolith.md)
- [0002 - Use PostgreSQL](./0002-use-postgresql.md)
- [0003 - Use REST between clients and API](./0003-use-rest-between-clients-and-api.md)
```

- O GitHub suporta links relativos em ficheiros Markdown no repositório.

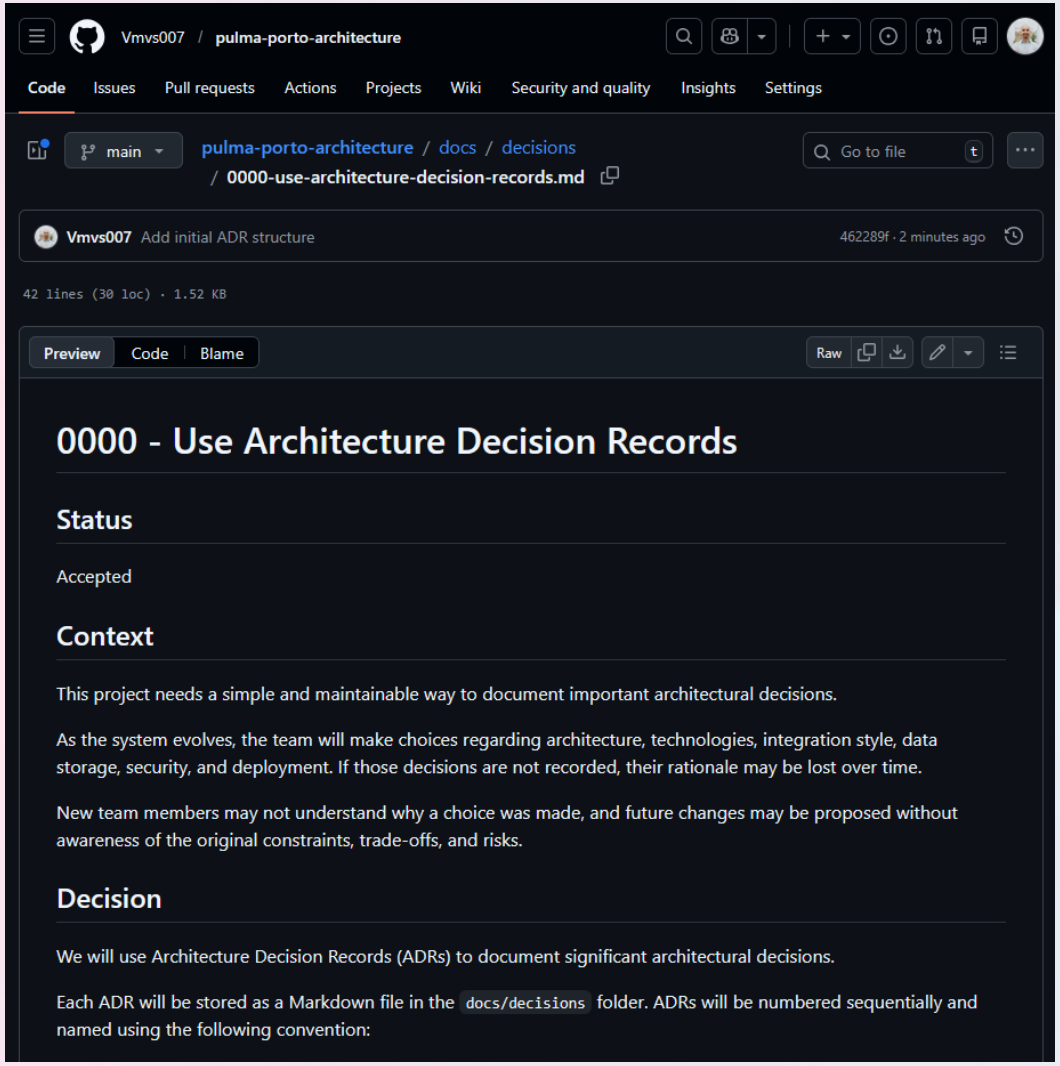
ADR – MELHORAR A NAVEÇÃO C/ ÍNDICE



ADR – MELHORAR A NAVEÇÃO C/ ÍNDICE



ADR – MELHORAR A NAVEÇÃO C/ ÍNDICE



ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

- Se quiser algo mais apresentável, é possível publicar a documentação com GitHub Pages.

  O GitHub Pages é um serviço de alojamento de sites estáticos que publica ficheiros do repositório, podendo usar a raiz do projeto ou a pasta /docs como fonte.

ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

- Como estamos a usar a pasta docs, é perfeito para isso.
  - No repositório, aceder a Settings
  - Abre Pages
  - Em Build and deployment → Em Source, escolhe Deploy from a branch
  - Seleciona:
    - Branch: main
    - Folder: /docs
  - Guarda
- O GitHub explica que é possível publicar a partir de uma branch e usar como origem a raiz do repositório ou a pasta /docs.

ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

- Para o GitHub Pages funcionar melhor, convém criar: docs/index.md
- Conteúdo:

```
---
title: Pulma Porto Architecture
---

# Pulma Porto Architecture

## Documentation

- [Architecture Decision Records](./decisions/)
```

ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

- Mudamos o README.md para corrigir os caminhos e aplicar alguma organização:

```
---
title: Architecture Decision Records
---

# Architecture Decision Records

- [0000 - Use Architecture Decision Records](./0000-use-architecture-decision-records.md)
- [0001 - Use Modular Monolith](./0001-use-modular-monolith.md)
- [0002 - Use PostgreSQL](./0002-use-postgresql.md)
- [0003 - Use REST Between Clients and API](./0003-use-rest-between-clients-and-api.md)
```

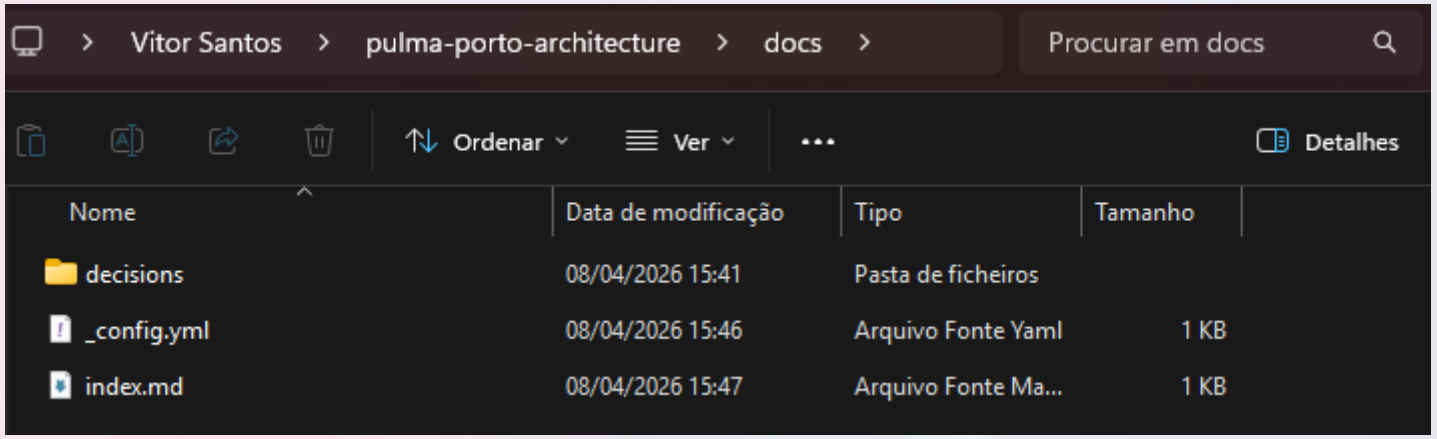
ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

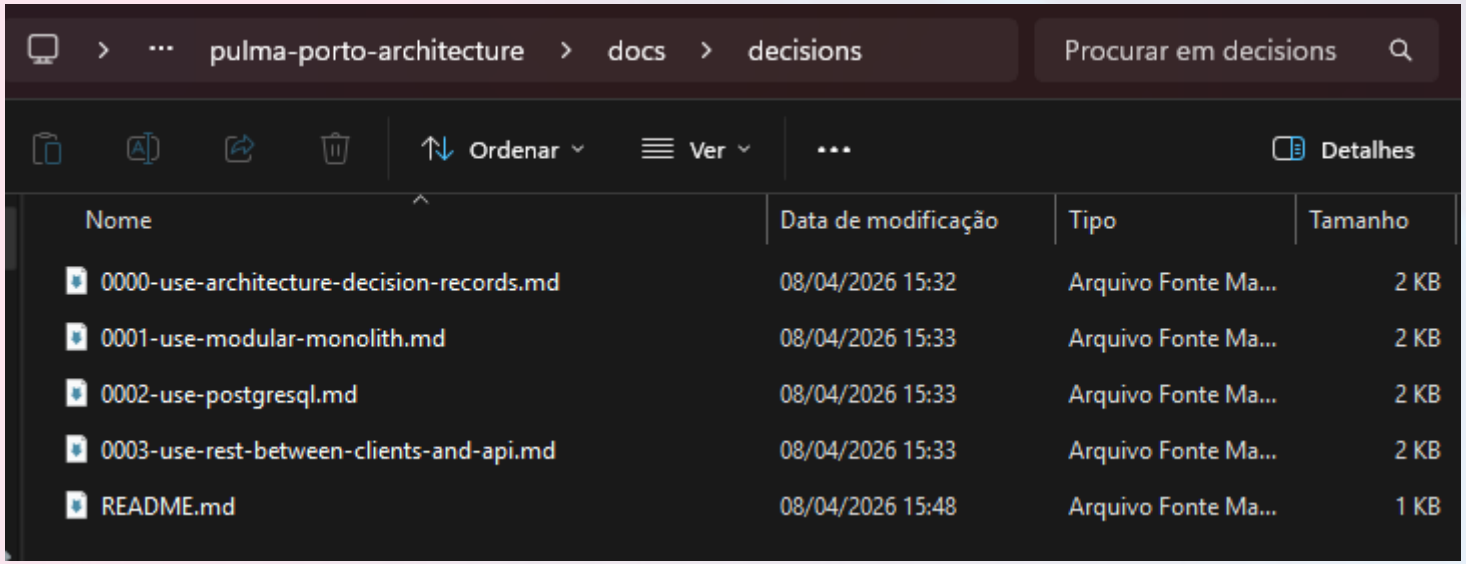
- Seguidamente, criamos o ficheiro docs/_config.yml
- Aqui podemos escolher o tema do Jekyll para personalizar a aparência da nossa página.
- Conteúdo:

```
theme: jekyll-theme-minimal
title: Pulma Porto Architecture
description: Architecture Decision Records and technical documentation
```

- Depois de o Pages publicar, vamos ter um site onde esta página é o ponto de entrada.
- Pode encontrar mais informações e temas em:
  - https://docs.github.com/pt/pages/setting-up-a-github-pages-site-with-jekyll/adding-a-theme-to-your-github-pages-site-using-jekyll

ADR – VISUALIZAR COMO SITE COM GITHUB PAGES





ADR – VISUALIZAR COMO SITE COM GITHUB PAGES



ADR – VISUALIZAR COMO SITE COM GITHUB PAGES

